

Final Model Performance Report

NPN_HACKATHON — Walmart M5 demand forecasting

Frozen-origin 28-day-ahead validation, 853,720 predictions

Shipped champion: $0.60 \times \text{Direct}(38f) + 0.40 \times \text{Recursive-shape}(32f)$

Project: NPN_HACKATHON — Walmart M5 store-item demand forecasting **Task:** frozen-origin 28-day-ahead forecasting

Scope of this document: reporting and verification only. No model was trained, changed or re-selected to produce it.

1. Validation setup

Item	Value
Forecast origin	d_1913 (2016-04-24)
Predicted days	d_1914 – d_1941 (2016-04-25 → 2016-05-22)
Horizon	28 days, generated in one shot from the origin
Series	30,490 store-item combinations
Predictions scored	853,720 (30,490 × 28)
Ground truth	real observed sales; held out, never used in training
Metric basis	validation only — no ground truth exists after d_1941

Every model in the comparison table below was scored on this identical set of 853,720 predictions, with no weighting and no series excluded. The protocol is strict: at origin *T* the model emits all 28 days, and nothing from *T+1...T+28* may enter any feature, model choice, calibration or blend weight.

2. Comparison table

Regression metrics (RMSE, MAE) are the task metrics. Accuracy / Precision / Recall / F1 are **classification-style demand-occurrence diagnostics**, defined in section 3 — they are not what any model was trained to optimise and must not be read as overall accuracy.

Model	Objective	RMSE	MAE	Accuracy (Demand > 0)	Precision	Recall	F1 Score
Naive - last value	none (arithmetic)	2.8936	1.3730	N/A	N/A	N/A	N/A
Naive - rolling mean 7	none (arithmetic)	2.2487	1.0683	N/A	N/A	N/A	N/A
Naive - rolling mean 28	none (arithmetic)	2.2430	1.0657	N/A	N/A	N/A	N/A
Naive - seasonal naive	none (arithmetic)	2.6769	1.2440	0.6570	0.6289	0.6027	0.6156
Global LightGBM (L2)	L2 (squared error)	2.1467	1.0411	0.6996	0.6360	0.7967	0.7073
Global LightGBM + Tweedie	Tweedie (p=1.1)	2.1256	1.0315	0.7039	0.6421	0.7913	0.7089
+ recency features	Tweedie (p=1.1)	2.1258	1.0320	0.7038	0.6415	0.7930	0.7093

Model	Objective	RMSE	MAE	Accuracy (Demand > 0)	Precision	Recall	F1 Score
Global LightGBM + Tweedie, 32 features *(original champion)*	Tweedie (p=1.1)	2.1210	1.0319	0.7038	0.6413	0.7939	0.7095
Hurdle (two-stage)	binary + Tweedie	2.1267	1.0324	0.7025	0.6393	0.7967	0.7093
Recursive one-step (member B)	Tweedie (p=1.1)	2.1182	1.0717	0.6897	0.6216	0.8148	0.7053
Shape only, 36 features	Tweedie (p=1.1)	2.1163	1.0299	0.7043	0.6420	0.7933	0.7097
Shape+Cycle, 38 features *(shape champion)*	Tweedie (p=1.1)	2.1157	1.0287	N/A	N/A	N/A	N/A
Diversity blend w=0.50 (38f + 26f)	Tweedie (p=1.1), both members	2.0920	1.0441	0.6967	0.6305	0.8076	0.7082
Diversity blend w=0.60 (38f + 32f) *(FINAL SHIPPED CHAMPION)*	Tweedie (p=1.1), both members	2.0929	1.0395	N/A	N/A	N/A	N/A

FINAL SHIPPED CHAMPION: 0.60 × Direct 38-feature model + 0.40 × Recursive-shape 32-feature model, RMSE **2.0929**, MAE **1.0395**.

Why four rows show N/A

N/A means the metric was **not evaluated**, never estimated. Occurrence metrics require per-row predictions, and four models do not have them on disk:

- **Naive last value / rolling mean 7 / rolling mean 28** — scored in Stage 1 from the registry; prediction files were not retained.
- **Shape+Cycle 38 features** — see the data-integrity note in section 6.
- **Diversity blend w=0.60 (the shipped model)** — Experiment #77 persisted metrics and per-window summaries but not per-row predictions. Computing its occurrence metrics would require retraining both members, which is a modelling run and out of scope here. The nearest saved artifacts are the w=0.50 blend and the 36-feature shape model, both reported above.

3. The demand-occurrence rule

A single rule, applied identically to every model with saved predictions:

```
actual event : y_true > 0      (at least one unit actually sold)
predicted event : y_pred >= 0.5 (point forecast rounds to >= 1 unit)
```

The 0.5 cut is the only non-arbitrary threshold for a count target: it is the value at which a forecast rounds to one unit. It was fixed once and never tuned per model.

Quantity	Value
Rows with actual demand > 0	388,995 of 853,720
Base rate	45.56%
Rows with actual demand = 0	464,725 (54.44%)

These models were never trained to classify. They minimise Tweedie deviance on a zero-inflated count target, so the occurrence metrics are a by-product of thresholding a regression output. A model could improve F1 while getting materially worse at the actual task.

4. Why RMSE and MAE are the primary metrics

The task is **how many units will sell**, not *whether any will*. Inventory decisions need the quantity: ordering 3 when 11 sell is a stockout, and both a "correct" occurrence classification.

- **RMSE** is the headline because it penalises large misses quadratically, and the business cost of demand error is convex — the expensive failures are the big ones. It is also the metric the whole campaign was pre-registered against.
- **MAE** is reported alongside because RMSE alone can be dominated by a small number of high-volume series. In this dataset the top volume decile carries **66%** of squared error, so RMSE and MAE can and do move in opposite directions. Reporting one without the other hides real trade-offs — including the one this project's final model makes.

Why there is no single valid "accuracy %"

- **The target is a count, not a class.** Accuracy needs a discretisation that the task does not supply; every threshold gives a different number.
- **A trivial model scores well.** 54.44% of rows are genuine zeros, so "always predict no demand" scores 54.44% accuracy while being useless. Our models reach ~70% — the honest comparison is against 54.44%, not 0%.
- **It discards magnitude entirely.** Predicting 1 when 40 sold counts as a correct positive.
- **It is threshold-dependent and therefore gameable.** Lowering the cut raises recall and accuracy on this class balance without improving any forecast.

The occurrence metrics are included because they were asked for and are genuinely informative about *one* aspect of behaviour — note the recursive member's distinctly high recall (0.8148) and low precision (0.6216) against the direct models, which is a real behavioural difference and part of why blending the two works. They are not a ranking metric.

5. Evidence behind the shipped champion

The headline 2.0929 is one window. The model was accepted on four windows and multiple seeds, with criteria fixed before each run.

Per-window, at the shipped weight $w = 0.60$

Window	Dates	RMSE	MAE	Δ RMSE vs direct	Δ MAE vs direct
primary_spring_2016	2016-04-25 → 05-22	2.0929	1.0395	-0.0281	+0.0102
christmas_2015	2015-12-12 → 2016-01-08	2.1547	0.9446	-0.0227	+0.0227
summer_2015	2015-07-16 → 08-12	2.1074	0.9909	-0.0187	+0.0224
autumn_2015	2015-10-02 → 10-29	2.1496	1.0126	-0.0274	+0.0189
Mean				-0.0242	+0.0186

High-volume decile (66% of all squared error)

Window	Direct champion	Shipped blend	Δ
primary_spring_2016	5.9787	5.8593	-0.1194
christmas_2015	6.3055	6.2195	-0.0861
summer_2015	6.2599	6.1813	-0.0786
autumn_2015	6.3767	6.2628	-0.1138

Improves on **every** window — the error concentration no earlier experiment managed to move.

Acceptance record

Experiment	Test	Result
#76	blend beats direct, 4 windows	4/4
#76	3 seeds, both members reseeded	3/3
#76	negative control (same architecture, reseeded)	-0.0044 vs -0.0291 — -0.0247 attributable to architecture
#77	member upgrade, 4 windows	4/4, mean -0.0042
#79	seed stability, 6 (window, seed) cells	6/6 blend, 6/6 member

Leakage was verified structurally at every window and at the forecast origin: all 38 direct-model features are bit-identical when every day after the origin is overwritten, and the recursive member's working matrix provably never contains post-origin actuals.

6. Data-integrity finding

The audit that produced this report re-derived every RMSE and MAE from the saved prediction files and compared them against the experiment registry. It failed on first run and surfaced a real defect:

`predictions/validation/exp_74_new_champion_validation.csv` is **not** the 38-feature champion's output. It is a byte-identical copy of `exp_72_shape_validation.csv` (same MD5), i.e. the **36-feature** shape model. Script `36_exp74_reproduce_and_extend.py` discards the 38-feature model's predictions at line 166 and writes Part A's at line 243.

What this does and does not affect:

- The champion's registry metrics (RMSE 2.1157 / MAE 1.0287) are **correct** — they were measured on the 38-feature model and have since been reproduced bit-identically (drift 0.00e+00).
- The mislabelled file was used as "the champion" in the headroom diagnostic that motivated Experiment #76. Its conclusion — that the recursive model is the standout blend partner — was then confirmed directly by retraining in #76 and #77, so nothing downstream rests on the mislabelled file.
- The 38-feature champion has **no** saved per-row predictions, which is why its occurrence metrics are N/A above.

Closing this properly requires one reproduction run to regenerate and correctly name that prediction file. That is a modelling run and was not performed for this report.

7. Non-comparable reference — the other team's reported result

The other team's reported RMSE $\approx$ 2.0324 is not comparable to any number in this report and must not be placed in the same table.

Their pipeline recomputes rolling and lag features *inside* the forecast horizon using actual sales from the validation window. Measured directly against their feature definitions (`experiments/artifacts/team_doc_analysis.json`):

Their feature	Days of the 28-day horizon that use future actuals
rolling_mean_7	27 / 28
rolling_mean_28	27 / 28
rolling_zero_count_7	27 / 28
lag_7	21 / 28
lag_28	0 / 28

A model that may read the answers for 27 of the 28 days it is predicting is solving a materially easier problem — closer to one-day-ahead forecasting with a rolling update than to 28-day-ahead forecasting from a frozen origin. Our own leakage probe, run deliberately and marked `DO_NOT_USE`, scored **1.9165** by permitting a similar violation; that figure is recorded in the ledger precisely so the gap is understood as a methodology difference and not a modelling one.

For a like-for-like view, `model_08_team_style_reproduction` implements their described approach under our frozen-origin rules and scores **2.1835** on the table above.

8. Known limitations of the shipped model

- **MAE regresses.** +0.0186 mean across four windows against the direct champion. Deliberate and disclosed: the blend is RMSE-optimal. The full weight frontier is in `exp77_operating_point.csv`; $w = 0.65$ trades -0.0234 RMSE for $+0.0158$ MAE if a different balance is wanted.
- **The #77 gain is concentrated.** Autumn (-0.0105) and Christmas (-0.0053) carry it; the primary and summer windows contributed -0.0005 and -0.0004 , which is noise. On summer the upgraded member was actually *worse* than the one it replaced.
- **Cost.** Two models instead of one — roughly double the training and inference budget, and the recursive member takes ~7 minutes to build because it retrains on 420 daily origins.
- **Seed evidence is partial.** #76's blend has a full 3-seed leg; #77's upgrade was seed-checked on the two windows that carry its effect (6/6 cells), not on all four.
- **The champion's 2.1157 is its most favourable draw.** Observed single-seed range was 2.1157–2.1211, so paired comparisons within a run are the honest ones, and this report quotes them that way in section 5.

9. Sources

Every figure traces to an artifact under version control:

Content	Path
Comparison table, as printed above	<code>experiments/artifacts/final_comparison_table.csv</code>
Same table plus audit columns and confusion counts	<code>experiments/artifacts/final_performance_comparison.csv</code>
Audit + table build	<code>scripts/06_research_campaign/44_final_performance_report.py</code>
Per-experiment records	<code>experiments/registry/*.json</code> (79 records)
Experiment index	<code>experiments/EXPERIMENT_LEDGER.md</code>
Blend acceptance	<code>exp_76_architectural_diversity_blend.json</code>
Member upgrade + weight frontier	<code>exp_77_recursive_member_upgrade.json</code> , <code>exp77_operating_point.csv</code>
Seed stability	<code>exp_79_upgrade_seed_check.json</code>
Leakage analysis of the team approach	<code>experiments/artifacts/team_doc_analysis.json</code>
Shipped forecast	<code>predictions/final_forecast/final_forecast_28day_v3_diversity_blend.csv</code>